MANUAL DE GIT
BLOQUE 1: FUNDAMENTOS Y ARQUITECTURA

Contenido

MANUAL DE GIT BLOQUE 1: FUNDAMENTOS Y ARQUITECTURA 1
1. ¿Qué es Git realmente? (Y por qué no es Dropbox) 2
2. La Santísima Trinidad de Git: Los Tres Estados 2
3. Configuración Inicial (Tu DNI) 3
4. Iniciando un Proyecto: git init 3
5. El Ciclo de Vida de un Archivo 3
Resumen de Comandos del Bloque 1 4
BLOQUE 2: EL FLUJO DE TRABAJO LOCAL 5
1. El GPS del Proyecto: git status 5
2. El Microscopio: git diff 5
3. El Portero de Discoteca: .gitignore 5
4. Deshacer cambios (El "Ctrl+Z" local) 6
5. Historial y Auditoría: git log 7
Resumen de Comandos del Bloque 2 7
BLOQUE 3: RAMAS Y FUSIONES (EL MULTIVERSO) 8
1. El Concepto: Universos Paralelos 8
2. Gestión de Ramas 9
A. Crear una rama 9
B. Moverse entre ramas (Switch) 9
C. Listar ramas 9
D. Borrar una rama 9
3. La Fusión: git merge 9
4. Conflictos: Cuando colisionan los mundos 10
Resumen de Comandos del Bloque 3 11
BLOQUE 4: TRABAJO COLABORATIVO Y REMOTO (GITHUB) 11
1. El Concepto: Local vs. Remoto 11
2. Conectando mundos: clone y remote 11
A. Empezar desde cero (Local -> Remoto) 11
B. Empezar desde algo existente (Remoto -> Local) 12
3. Sincronización: Subir y Bajar 12
A. Subir cambios: git push 12
B. Bajar cambios: git pull 12
4. El Flujo Profesional: Pull Requests (PR) 12
5. Fork vs. Clone (La duda eterna) 13
Resumen de Comandos del Bloque 4 13
BLOQUE 5: HERRAMIENTAS DE PÁNICO Y BUENAS PRÁCTICAS 14
1. El Cajón de Sastre: git stash 14
2. El Botón de Deshacer: Reset vs Revert 14
3. La Opción Nuclear: Sincronización Forzosa 15
Resumen de Comandos del Bloque 5 16

1. ¿Qué es Git realmente? (Y por qué no es Dropbox)

Imagina que estás escribiendo un documento importantísimo. Normalmente harías esto:

- trabajo.docx
- trabajo_final.docx
- trabajo_final_bueno.docx
- trabajo_final_ESTE_SI_QUE_SI.docx

Esto es un desastre. **Git** viene a solucionar esto. Git es una **Máquina del Tiempo**. No guarda copias de tus archivos con nombres distintos. Lo que hace es tomar **fotografías instantáneas (Snapshots)** de todo tu proyecto cada vez que tú se lo pides.

Permite:

1. Viajar al pasado (ver cómo estaba el código hace 3 días).
2. Crear universos paralelos (intentar una idea loca sin romper lo que ya funciona).
3. Fusionar el trabajo de varias personas sin que se sobrescriban unos a otros.

💡 **OJO:** Git **NO es GitHub**.

- **Git:** Es el motor del coche (el software que instalas en tu PC).
- **GitHub:** Es el garaje en la nube donde aparcas el coche (una web que aloja tus copias de Git).

---

2. La Santísima Trinidad de Git: Los Tres Estados

Para entender Git, tenéis que grabaros a fuego este esquema. Un archivo en Git puede vivir en tres zonas diferentes:

A. Working Directory (Tu mesa de trabajo)

Es la carpeta que ves en Windows/Mac. Aquí es donde editas, borras y creas archivos. Es el "presente sucio".

- *Estado:* **Modified / Untracked**.
- *Situación:* Los cambios están en tu disco duro, pero Git aún no los ha "guardado" oficialmente en su historia.

B. Staging Area (El estudio de fotografía)

También llamada "Index". Es una zona intermedia. Aquí colocas los archivos que quieres que salgan en la próxima foto.

- *Estado:* **Staged**.
- *Analogía:* Imagina que vas a hacer una foto de grupo. El *Working Directory* es la fiesta (caos). El *Staging Area* es cuando dices: "Eh, tú, tú y tú, poneos delante de la cámara y sonreíd". Aún no has disparado la foto, pero ya están preparados.

C. Repository / .git directory (El álbum de fotos)

Es donde Git guarda las fotos definitivas. Una vez que algo entra aquí, es muy difícil perderlo.

- *Estado:* **Committed**.
- *Situación:* La versión está guardada para siempre en la base de datos de Git con un código único (Hash).

---

3. Configuración Inicial (Tu DNI)

Antes de hacer nada, Git necesita saber quién eres para firmar tus fotos. Si no haces esto, te dará la lata después.

Abre la terminal y ejecuta (una sola vez por ordenador):

```
# Dile a Git cómo te llamas
git config --global user.name "Tu Nombre Real"


# Dile a Git tu email (IMPORTANTE: Usa el mismo que usarás en GitHub)
git config --global user.email "tu_email@ejemplo.com"
```

- --global: Significa que esta configuración vale para todos los proyectos de tu ordenador.
- Para ver si lo has hecho bien: git config --list

---

4. Iniciando un Proyecto: git init

¿Cómo transformamos una carpeta normal en un proyecto con superpoderes Git?

1. Crea una carpeta nueva o entra en una existente.
2. Ejecuta:

```
git init
```

¿Qué acaba de pasar? Si activas la opción de "Ver archivos ocultos" en tu sistema, verás que ha aparecido una carpeta fantasma llamada .git.

- **¡PELIGRO!** 💀 Nunca, bajo ningún concepto, toques, borres o entres en la carpeta .git a menos que sepas *muy bien* lo que haces. Ahí reside el cerebro de tu proyecto. Si la borras, pierdes toda la historia y vuelves a tener una carpeta normal.

---

5. El Ciclo de Vida de un Archivo

Vamos a ver cómo se mueven los archivos entre las tres zonas que explicamos antes.

Paso 1: Crear un archivo (Working Directory)

Creas index.html. Git se da cuenta, pero te dice que es **Untracked** (No está en seguimiento). Git te está diciendo: *"Veo un archivo nuevo, pero no es responsabilidad mía hasta que tú me lo digas"*.

Paso 2: Prepararlo (git add) -> Al Staging Area

Queremos incluir este archivo en la próxima foto.

```
git add index.html
```

Ahora el archivo está en el **Staging Area**. Está "listo para foto".

- *Truco:* Si quieres añadir TODO lo que has cambiado de golpe, usa git add .

Paso 3: Confirmarlo (git commit) -> Al Repositorio

Disparamos la cámara. Guardamos la versión.

```
git commit -m "Creación de la página de inicio"
```

- -m "...": Es el mensaje que explica QUÉ has hecho.
- **Regla de oro:** El mensaje debe ser descriptivo.
    - Mal: "Cambios", "asdf", "arreglo".
    - Bien: "Añadido botón de login", "Corregido error en el menú".

Una vez hecho el commit, el *Staging Area* se vacía. Tu *Working Directory* está "limpio" (clean).

Resumen de Comandos del Bloque 1

Comando	Descripción	Zona afectada
git init	Crea el repositorio (carpeta .git).	-
git config ...	Configura tu nombre y email.	-
git status	EL MÁS IMPORTANTE. Te dice cómo están tus archivos (Rojos = Working, Verdes = Staging).	Todas
git add <archivo>	Pasa del Working Directory al Staging Area.	Working -> Staging
git commit -m "txt"	Guarda lo que hay en Staging en el Historial permanentemente.	Staging -> Repo
git log	Muestra el historial de "fotos" (commits) que has hecho.	Repo

BLOQUE 2: EL FLUJO DE TRABAJO LOCAL

Ya tienes tu repositorio iniciado. Pero el software no es estático; vas a estar editando código constantemente. En este bloque aprenderemos a gestionar los cambios y a mantener el proyecto limpio.

---

1. El GPS del Proyecto: git status

Si te despiertan a las 3 de la mañana y te preguntan un comando de Git, tu respuesta debe ser git status.

Este comando te dice la verdad absoluta sobre tu proyecto.

- **Archivos Rojos:** Cambios que has hecho pero que Git aún no ha "preparado" (Modified).
- **Archivos Verdes:** Cambios que ya están en el *Staging Area*, listos para el próximo commit.
- **Archivos Untracked:** Archivos nuevos que Git no conoce.

💡 **Consejo:** Ejecuta git status antes de cada add y antes de cada commit. Evitará que subas cosas que no querías.

---

2. El Microscopio: git diff

A veces git status se queda corto. Te dice: *"Has modificado index.html"*. Vale, pero... ¿qué he cambiado? ¿He borrado algo importante?

Para eso existe git diff.

- **git diff (a secas):** Compara tu *Working Directory* (lo que tienes abierto ahora) con el *Staging Area* (o el último commit si el staging está vacío).
    - Te muestra en **Rojo** (-) lo que has borrado.
    - Te muestra en **Verde** (+) lo que has añadido.
- **git diff --staged:** Te muestra los cambios que ya has hecho add y que van a ir en el próximo commit. Útil para revisar justo antes de disparar la foto.

---

3. El Portero de Discoteca: .gitignore

En todo proyecto hay archivos que **NUNCA** deben subirse a Git. Jamás.

- **Archivos de dependencias pesadas:** node_modules (Node), vendor (PHP/Composer). Pesan cientos de megas y se pueden reinstalar.
- **Archivos con claves secretas:** .env (Contraseñas de BBDD, API Keys). Si subes esto, los hackers (o Javier, el de Ciberseguridad) te crujirán.

- **Archivos del sistema:** .DS_Store (Mac), Thumbs.db (Windows).

¿Cómo se hace?

Creas un archivo en la raíz de tu proyecto llamado (exactamente así) .gitignore. Es un archivo de texto simple. Todo lo que escribas ahí, Git lo ignorará y hará como que no existe.

Ejemplo típico para vuestros proyectos:

Plaintext

/vendor

/node_modules

.env

.DS_Store

⚠️ **Importante:** Si ya has subido un archivo (hiciste commit) y LUEGO lo añades al .gitignore, Git **NO** lo ignorará. Tienes que borrarlo del repositorio primero (git rm --cached archivo). El .gitignore solo funciona con archivos que aún no han entrado en la historia.

---

4. Deshacer cambios (El "Ctrl+Z" local)

A todos nos pasa. Escribes código, lo rompes todo y dices: *"Ojalá pudiera volver a como estaba el archivo hace 10 minutos"*.

Git tiene comandos para esto, dependiendo de en qué fase estés (mira el diagrama del Bloque 1).

CASO A: He editado un archivo, pero aún no he hecho git add

El archivo está modificado (Rojo). Quiero descartar esos cambios y dejarlo como estaba en el último commit.

- **Comando Moderno (Git 2.23+):**

```
git restore nombre-archivo.php
```

- **Comando Clásico:**

```
git checkout -- nombre-archivo.php
```

- **Efecto:** Pierdes lo que acabas de escribir. El archivo vuelve a su estado "seguro".

CASO B: He hecho git add, pero me he arrepentido (aún no hice commit)

El archivo está en el Staging Area (Verde). Quiero sacarlo de ahí, pero **no quiero perder mis cambios**, solo quiero que no entren en la foto todavía.

- **Comando Moderno:**

```
git restore --staged nombre-archivo.php
```

- **Comando Clásico:**

```
git reset HEAD nombre-archivo.php
```

- **Efecto:** El archivo pasa de Verde a Rojo. Tus cambios siguen ahí, pero ya no están listos para commit.

---

5. Historial y Auditoría: git log

Cuando lleves un mes trabajando, querrás ver qué hiciste.

```
git log
```

Te muestra una lista aburrida y larga.

El comando PRO que debéis usar:

```
git log --oneline --graph –all
```

- --oneline: Resumen en una línea por commit.
- --graph: Dibuja un arbolito con líneas de colores (útil cuando veamos ramas).
- --all: Muestra todas las ramas, no solo la actual.

Aquí veréis el famoso **HASH** (ese código raro tipo a1b2c3d). Es la matrícula única de cada commit. Lo usaréis para viajar en el tiempo (lo veremos en el Bloque 5).

Resumen de Comandos del Bloque 2

Situación	Comando
Ver qué está pasando	git status
Ver qué líneas han cambiado	git diff
Ver qué voy a commitear	git diff --staged
Ignorar archivos	Crear archivo .gitignore y escribir nombres dentro.
Descartar cambios locales (Resetear archivo)	git restore <archivo>
Sacar del "limbo" (Unstage)	git restore --staged <archivo>
Ver historial bonito	git log --oneline --graph --all

BLOQUE 3: RAMAS Y FUSIONES (EL MULTIVERSO)

Hasta ahora, hemos trabajado de forma lineal: cambio A -> cambio B -> cambio C. Pero la vida real no es lineal.

Si todos trabajáramos sobre el mismo archivo a la vez, se sobrescribirían. Aquí es donde entra el concepto de Ramas (Branches).

¡Allé vamos! Aquí entra la verdadera magia de Git.

Hasta ahora, tus alumnos han trabajado de forma lineal: cambio A -> cambio B -> cambio C. Pero la vida real no es lineal. En un equipo, **David** está corrigiendo un examen, **Javier** está hackeando el servidor y **Lidia** está cambiando el CSS, todo al mismo tiempo.

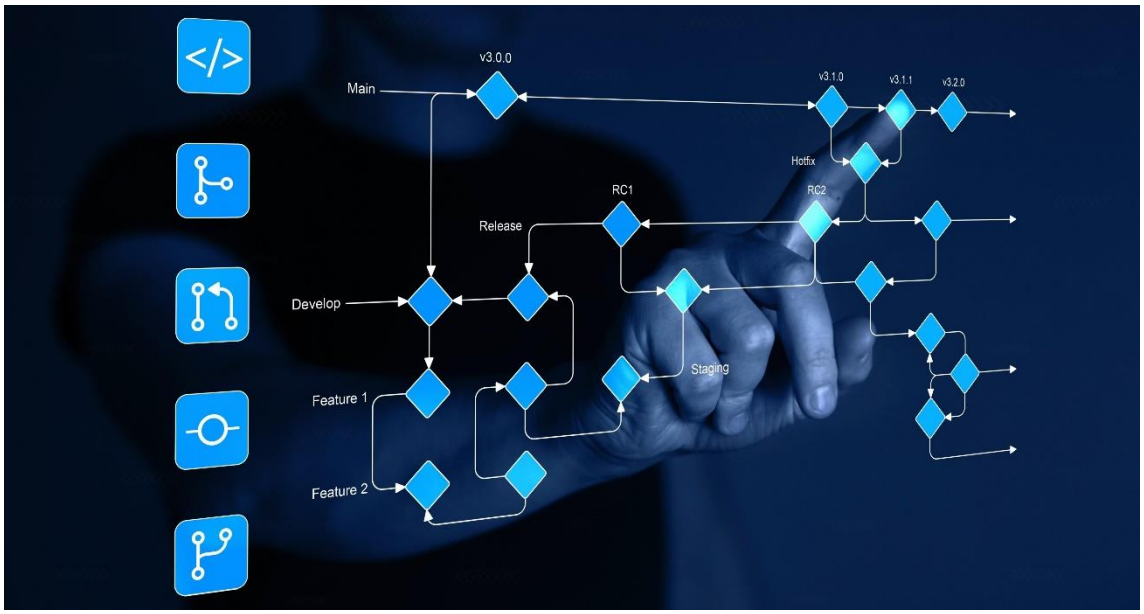
Si todos trabajaran sobre el mismo archivo a la vez, se sobrescribirían. Aquí es donde entra el concepto de **Ramas (Branches)**.

1. El Concepto: Universos Paralelos

Imagina que estás jugando a un videojuego y llegas a un jefe difícil. Antes de entrar, guardas la partida (Save Game).

- Si el jefe te mata, recargas la partida y lo intentas de otra forma.
- Si le ganas, sigues adelante.

En Git, la "partida guardada" es la rama principal, llamada históricamente master, pero que ahora llamamos **main**. Una **Rama (Branch)** es crear una línea temporal alternativa. Puedes romper todo lo que quieras en esa rama; el main (la realidad original) seguirá intacta y funcionando.



2. Gestión de Ramas

Por defecto, estás en la rama main. Es la rama sagrada. **Regla de oro:** Nunca se programa directamente en main.

A. Crear una rama

Quieres añadir un formulario de contacto. Creas una rama llamada feature/contacto.

```
git branch feature/contacto
```

Esto crea la rama, **pero NO te mueve a ella**. Sigues en main.

B. Moverse entre ramas (Switch)

Para "saltar" a esa línea temporal:

```
git checkout feature/contacto
```

(Nota: En versiones muy nuevas de Git veréis git switch, pero checkout es el clásico que veréis en todos los tutoriales).

🚀 **El Truco Pro:** Crear y saltar a la vez (el comando que usaréis el 99% de las veces):

```
git checkout -b feature/contacto
```

C. Listar ramas

¿Dónde estoy? ¿Cuántas ramas hay?

```
git branch
```

Git te mostrará una lista. La que tiene el asterisco * y está en verde es **donde estás pisando ahora mismo**.

D. Borrar una rama

Ya has terminado con la rama y la has fusionado (o era una idea terrible y quieres quemarla).

```
git branch -d nombre-rama
```

(Si Git se queja porque no la has fusionado y quieres borrarla a la fuerza, usa -D mayúscula).

---

3. La Fusión: git merge

Has trabajado en tu rama feature/contacto, todo funciona bien y quieres llevar esos cambios a la rama main (la realidad oficial). Esto es el **Merge**.

⚠️ **IMPORTANTE:** Para fusionar, **siempre debes colocarte primero en la rama de DESTINO**. Si quieres llevar cambios *hacia* main, ponte *en* main.

1. git checkout main (Vuelves a la base).
2. git merge feature/contacto (Traes los cambios de la otra rama hacia ti).

Aquí pueden pasar dos cosas:

Escenario A: Fast-Forward (El camino fácil)

Desde que creaste tu rama, **nadie ha tocado el main**. Git dice: *"Ah, fácil. Simplemente muevo la etiqueta de Main hasta donde estás tú"*. Es lineal. No crea ningún "nudo" en la historia.

Escenario B: Merge Commit (El nudo de corbata)

Mientras tú trabajabas en tu rama, **alguien (ej. Javier) subió cambios a main**. Ahora las historias han divergido. Git dice: *"Vale, tengo que juntar lo que hizo Javier con lo que hiciste tú"*. Git creará automáticamente un **nuevo commit de unión** (Merge Commit). Verás que se abre un editor para poner el nombre del commit.

---

4. Conflictos: Cuando colisionan los mundos

A veces, Git no puede fusionar solo. Esto ocurre cuando **dos personas han tocado la misma línea del mismo archivo** de forma diferente.

Ejemplo:

- En main, el archivo dice: color: blue;
- En tu rama, cambiaste esa línea a: color: red;

Al intentar el merge, Git grita: CONFLICT (content): Merge conflict in estilos.css

Git detiene el proceso y te dice: *"Yo no decido. Decide tú, humano"*.

Anatomía de un Conflicto

Si abres el archivo en conflicto, verás esto:

```
<<<<<<< HEAD
color: blue;
=======
color: red;
>>>>>>> feature/contacto
```

- **<<<<<<< HEAD**: Lo que había en la rama donde estás (Main).
- **=======**: La frontera.
- **>>>>>>> feature/...**: Lo que viene de la rama que intentas fusionar.

¿Qué hacer?: Borrar los símbolos raros y dejar el código final correcto. Luego hacer git add y git commit para sellar la paz.

Resumen de Comandos del Bloque 3

Acción	Comando
Crear una rama	git branch <nombre>
Cambiar a una rama	git checkout <nombre>
Crear y cambiar (Combo)	git checkout -b <nombre>
Ver todas las ramas	git branch
Fusionar (estando en destino)	git merge <rama-origen>
Borrar rama	git branch -d <nombre>

BLOQUE 4: TRABAJO COLABORATIVO Y REMOTO (GITHUB)

Este es el bloque crítico para vuestra práctica en equipos. Hasta ahora, Git era una herramienta solitaria. Ahora conectamos el cable a internet y entramos en GitHub. Aquí es donde ocurren el 90% de los problemas de "no me deja subir" y donde se organiza el caos.

1. El Concepto: Local vs. Remoto

Hasta ahora, tu repositorio vivía solo en tu disco duro. Si tu ordenador explotaba, adiós proyecto. Un **Remoto** es simplemente una copia de tu proyecto alojada en un servidor en internet (GitHub, GitLab, Bitbucket).

- **Local:** Tu ordenador. Donde trabajas.
- **Remoto:** La nube. Donde se centraliza el trabajo de todos.
- **Origin:** Es el nombre cariñoso (alias) que Git le da por defecto a la URL de tu repositorio en GitHub.

---

2. Conectando mundos: clone y remote

Hay dos formas de empezar a trabajar con un servidor remoto:

A. Empezar desde cero (Local -> Remoto)

Tienes código en tu PC y quieres subirlo a un repo nuevo vacío en GitHub.

```
git remote add origin https://github.com/usuario/proyecto.git
```

B. Empezar desde algo existente (Remoto -> Local)

Quieres descargarte el proyecto de un compañero o del profesor.

```
git clone https://github.com/usuario/proyecto.git
```

- **¿Qué hace clone?** Descarga el proyecto completo (con todo su historial) y configura automáticamente el remoto origin. ¡Solo se hace una vez al principio!

---

3. Sincronización: Subir y Bajar

Una vez conectados, el código no viaja solo por telepatía. Tienes que empujarlo o tirar de él explícitamente.

A. Subir cambios: git push

Has hecho tus commits en local. Ahora quieres enviarlos a la nube.

```
git push origin main
```

- Traducido: *"Git, empuja (push) lo que tengo en mi rama local hacia la rama 'main' del remoto 'origin'"*.

> 💡 **Nota:** La primera vez que subas una rama nueva, Git te pedirá que la vincules. Usa: git push -u origin nombre-rama

B. Bajar cambios: git pull

Tu compañero te dice: *"Ya he subido el código arreglado"*. Tú no lo tienes en tu PC. Necesitas bajarlo.

```
git pull origin main
```

- **¿Qué hace realmente?** Es un comando "2 en 1".
    1. Descarga los datos (git fetch).
    2. Fusiona automáticamente esos datos con tu rama actual (git merge).

---

4. El Flujo Profesional: Pull Requests (PR)

En equipos serios (y en vuestra práctica), **está prohibido hacer push directo a main**. ¿Por qué? Porque si subes un error, rompes la web para todos.

El flujo correcto es el **Pull Request (Solicitud de Fusión)**.

1. **Crea una rama:** Trabaja en feature/mi-idea.
2. **Sube tu rama:** git push origin feature/mi-idea (No subes a main, subes a tu rama en la nube).
3. **Ve a GitHub:** Al entrar en la web, verás un botón amarillo/verde que dice "Compare & pull request".
4. **Abre el PR:** Escribe qué has hecho y dale a crear.

5. **Revisión (Code Review):** Tus compañeros ven tu código en la web, dejan comentarios y, si todo está bien, le dan al botón **Merge**.

OJO: El Pull Request **NO es un comando de terminal**. Es una funcionalidad de la página web de GitHub.

---

5. Fork vs. Clone (La duda eterna)

En la práctica usaremos el concepto de **Fork**. Es vital entender la diferencia.

- **CLONE (Clonar):** Haces una copia en tu PC de un repo al que tienes acceso. Si haces push, escribes directamente en el repo original.
    - *Uso:* Trabajadores de la misma empresa con permisos.
- **FORK (Bifurcar):** Haces una copia **en tu cuenta de GitHub** de un proyecto ajeno (ej. el del profesor). Tienes control total sobre TU copia, pero no tocas el original del profesor.
    - *Uso:* Open Source, o alumnos que no quieren romper el código base del profesor.
    - *Flujo:* Fork del repo del profe -> Clone de TU Fork -> Trabajas -> Push a TU Fork -> Pull Request al repo del profe.

Resumen de Comandos del Bloque 4

Acción	Comando
Descargar un repo por primera vez	git clone <url>
Ver a dónde está conectado mi repo	git remote -v
Subir mis commits a la nube	git push origin <rama>
Bajar cambios de la nube y fusionar	git pull origin <rama>
Bajar cambios SIN fusionar (solo mirar)	git fetch
Pull Request	¡Se hace en el navegador web!

BLOQUE 5: HERRAMIENTAS DE PÁNICO Y BUENAS PRÁCTICAS

Este es el broche final. Es el bloque que convierte a un principiante asustadizo en un usuario confiado. Aquí están las herramientas para salir del barro cuando todo parece perdido y las normas de etiqueta para no ser "ese compañero que rompe el repo".

En el mundo real, te vas a equivocar. Vas a borrar archivos sin querer, vas a trabajar en la rama equivocada o vas a subir código roto. No pasa nada. Git está diseñado para perdonar (casi) todo.

---

1. El Cajón de Sastre: git stash

Situación: Estás programando una funcionalidad a medias (tienes archivos rojos y verdes), todo está sucio... y de repente te dice el jefe (o David): *"¡Deja eso! Hay un error crítico en producción, cámbiate a main y arréglalo YA"*.

No puedes cambiar de rama porque Git te dirá que perderás tu trabajo. Pero tampoco quieres hacer un commit de algo a medio hacer ("código roto, luego lo arreglo").

Solución: Guardarlo en el "limbo" temporal.

1. **Guardar todo (Esconder):**

```
git stash
```

Tu mesa de trabajo se limpia mágicamente. Vuelves al estado del último commit. Ahora puedes cambiar de rama tranquilo.

2. **Recuperar lo guardado:** Cuando vuelvas a tu rama y quieras seguir:

```
git stash pop
```

Tus cambios reaparecen tal cual los dejaste.

---

2. El Botón de Deshacer: Reset vs Revert

Esta es la pregunta del millón en las entrevistas de trabajo. Ambos sirven para "volver atrás", pero funcionan de forma opuesta.

A. git reset (La Máquina del Tiempo)

Borra la historia. Vuelves al pasado como si los commits posteriores nunca hubieran existido.

- **Peligro:** Si haces esto en una rama compartida (main), tus compañeros te odiarán porque sus historias y la tuya dejarán de coincidir.
- **Uso:** Solo para tu rama local, cuando te has equivocado en tus últimos commits y nadie los ha visto aún.

Tipos de Reset:

1. **Soft (Suave):** `git reset --soft HEAD~1`
    - Deshace el último commit, pero **mantiene tus archivos modificados** (en verde/staged).
    - *Para qué sirve:* "Me equivoqué en el mensaje del commit" o "Se me olvidó meter un archivo en el commit".
2. **Hard (Duro):** `git reset --hard HEAD~1`
    - **Destrucción total.** Borra el commit y borra tus cambios en los archivos.
    - *Para qué sirve:* "He hecho un desastre total, quiero volver a como estaba ayer".

B. git revert (La Corrección Pública)

No borra nada. Crea un **NUEVO commit** que hace exactamente lo contrario al commit que quieres anular.

- **Seguridad:** Es seguro para usar en main. La historia avanza, no retrocede.
- **Ejemplo:** Si el commit A escribió "Hola", el Revert del commit A escribirá "Adiós" (o borrará la línea).

Comando:

```
git revert <hash-del-commit-culpable>
```

---

3. La Opción Nuclear: Sincronización Forzosa

A veces, tu repositorio local está tan roto o desincronizado que no vale la pena arreglarlo. Solo quieres que sea idéntico a lo que hay en GitHub (Main).

El comando que te salva la vida (y que usamos antes):

```
git fetch origin
git reset --hard origin/main
```

Traducción: "Tira a la basura todo lo mío y ponme lo que haya en el servidor".

---

4. Los 10 Mandamientos de Git (Buenas Prácticas)

Para aprobar el curso (y sobrevivir en una empresa), seguid estas reglas:

1. **No commitearás código roto:** Si la web no carga, no hagas commit. Usa stash.
2. **Harás commits atómicos:** Un commit debe hacer **una** sola cosa.
    - Mal: *"Arreglado el login, cambiado el color del footer y añadida la foto de Javier"*.
    - Bien: Tres commits separados. Si hay que deshacer el color del footer, no nos cargamos el login.

3. **Escribirás mensajes útiles:** Imagina que lees el historial dentro de un año. "Fix" no ayuda. "Corregido error 404 en perfil" sí.
4. **Nunca trabajarás en main:** Siempre crea una rama (feature/... o fix/...).
5. **Harás git pull antes de empezar:** Antes de escribir una sola línea de código por la mañana, actualízate.
6. **No subirás secretos:** Revisa el .gitignore antes de subir claves o contraseñas.
7. **Revisarás tu propio código:** Haz un git diff antes de commitear. A veces dejamos console.log o comentarios basura.

Resumen de Comandos del Bloque 5

Situación	Comando
Guardar trabajo a medias	git stash
Recuperar trabajo guardado	git stash pop
Deshacer último commit (manteniendo archivos)	git reset --soft HEAD~1
Deshacer último commit (borrando todo)	git reset --hard HEAD~1
Anular un commit antiguo (seguro para main)	git revert <hash>
Ver historial de una sola línea de un archivo	git blame <archivo>